MODULE 3

Classification Techniques

Introduction to Machine Learning

Complete Study Notes

From first principles to professional practice

Course instructor
Prof. Rajesh Nag

Topics covered

Logistic Regression • k-Nearest Neighbours (KNN) • Naïve Bayes • Confusion matrix • Accuracy, precision and recall

Worked examples, diagrams, code, exam tips and a revision toolkit included.

How to Use These Notes

Start with the green *In Plain Words* boxes if the topic is new to you; learn the red *Definition* boxes and the *Worked Examples* for exams; read the purple *Pro Corner* boxes for professional depth.

Box	What it gives you
In Plain Words	Everyday analogy; the idea with no jargon
Definition / Key Idea	Exact wording to write in exams
Worked Example	Step-by-step calculation
Tip	Helpful shortcuts and intuition
Watch Out	Common mistakes and traps
Exam Tip	How questions are usually asked and answered
Pro Corner	Deeper, professional-level details

Suggested study plan: read a chapter once, redo every worked example without looking, then attempt the practice questions in Chapter 6. The numerical questions most often come from KNN (distance and voting), Naïve Bayes (posterior probability) and the confusion matrix (accuracy, precision, recall).

Contents

How to Use These Notes i

1 Introduction to Classification 1
- 1.1 Kinds of Classification Problems . . . 1
- 1.2 How a Classifier Works . . . 1
- 1.3 Useful Ways to Compare Classifiers . . . 2

2 Logistic Regression 3
- 2.1 Why Not Just Use Linear Regression? . . . 3
- 2.2 The Sigmoid Function . . . 3
- 2.3 The Model . . . 3
- 2.4 Odds and Log-Odds: Interpreting the Coefficients . . . 4
- 2.5 Learning the Weights: Log-Loss (Cross-Entropy) . . . 4
- 2.6 Choosing the Threshold . . . 4
- 2.7 Multiclass Logistic Regression . . . 4
- 2.8 Regularisation . . . 5
- 2.9 Assumptions, Strengths and Weaknesses . . . 5
- 2.10 Code . . . 5

3 k-Nearest Neighbours (KNN) 7
- 3.1 The Algorithm . . . 7
- 3.2 Distance Measures . . . 7
- 3.3 Worked Example . . . 8
- 3.4 Choosing k . . . 8
- 3.5 Weighted KNN . . . 9
- 3.6 Feature Scaling is Essential . . . 9
- 3.7 Complexity . . . 9

3.8 Strengths and Weaknesses . . . 9
3.9 Code . . . 9

4 Naïve Bayes **11**
4.1 Bayes' Theorem . . . 11
4.2 The Naïve Assumption . . . 11
4.3 Worked Example (Spam Filter) . . . 12
4.4 The Zero-Frequency Problem and Laplace Smoothing . . . 12
4.5 Variants of Naïve Bayes . . . 12
4.6 Practical Tips . . . 13
4.7 Strengths and Weaknesses . . . 13
4.8 Code . . . 13

5 Confusion Matrix, Accuracy, Precision and Recall **15**
5.1 The Confusion Matrix . . . 15
5.2 Accuracy, Precision and Recall . . . 15
5.3 Worked Example . . . 16
5.4 The Accuracy Paradox . . . 17
5.5 More Metrics from the Same Matrix . . . 17
5.6 The Precision–Recall Trade-off and the Threshold . . . 17
5.7 Multiclass Confusion Matrix . . . 18
5.8 Which Metric Should I Use? . . . 19
5.9 Code . . . 19

6 Comparison and Revision Toolkit **21**
6.1 Logistic Regression vs. KNN vs. Naïve Bayes . . . 21
6.2 One-Page Summary . . . 21
6.3 Formula Sheet . . . 22
6.4 Commonly Confused Pairs . . . 22
6.5 Short-Answer Practice . . . 22
6.6 Long-Answer Outlines . . . 23
6.7 Multiple-Choice Practice . . . 23
6.8 Glossary . . . 24

1 Introduction to Classification

In Plain Words

A doctor looks at a patient's symptoms and decides: "flu", "cold" or "allergy". A bank officer looks at an application and decides: "approve" or "reject". In both cases the answer is a *category*, not a number. Teaching a computer to make such category decisions from past examples is **classification**.

Definition: Classification

A supervised learning task in which the model learns from labelled examples $\{(x_i, y_i)\}$, where each label y_i belongs to a **finite set of classes**, and then predicts the class of new, unseen inputs x.

1.1 Kinds of Classification Problems

Kind	Meaning	Example
Binary	Exactly two classes (positive / negative)	Spam or not spam; disease or healthy
Multiclass	More than two classes, exactly one is correct	Digit 0–9; flower species
Multilabel	Several labels can be true at once	A photo tagged "beach" and "sunset"
Imbalanced	One class is far rarer than the others	Fraud (0.1% of transactions)

1.2 How a Classifier Works

1. **Training:** the algorithm studies labelled data and builds a model.
2. **Prediction:** for a new input, the model outputs a class (or class probabilities).
3. **Evaluation:** we compare predictions with true labels on unseen test data (Chapter 5).

Decision boundary. Imagine plotting samples by two features. A classifier effectively draws a boundary separating the regions of each class. A new point is labelled by the side it falls on. Logistic regression draws a straight line; KNN draws a flexible, jagged boundary; Naïve Bayes draws a smooth curve or line depending on its variant.

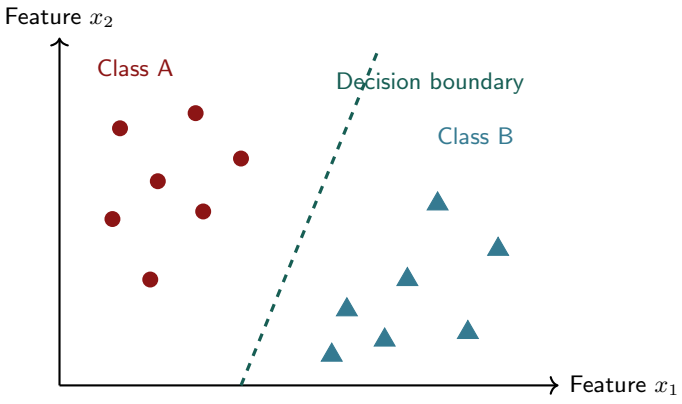


Figure 1. A classifier separates the feature space into class regions.

1.3 Useful Ways to Compare Classifiers

Axis	Option A	Option B
Learning style	**Eager:** builds a model during training, predicts fast (logistic regression, Naïve Bayes)	**Lazy:** stores data, does the work at prediction time (KNN)
Probabilistic view	**Discriminative:** learns $P(y \mid x)$ directly (logistic regression)	**Generative:** learns $P(x \mid y)$ and $P(y)$, then uses Bayes' rule (Naïve Bayes)
Parameters	**Parametric:** fixed number of parameters (logistic regression, Naïve Bayes)	**Non-parametric:** grows with data (KNN)
Output	**Hard label:** just the class	**Soft output:** class probabilities (useful for thresholds)

Exam Tip

Be ready to say, in one line each: logistic regression is a *discriminative, parametric, eager* classifier; KNN is *lazy, instance-based, non-parametric*; Naïve Bayes is *generative, probabilistic, eager*.

2 Logistic Regression

In Plain Words

Despite the word "regression", logistic regression is a *classification* method. It answers: "**what is the probability that this sample belongs to the positive class?**" (for example, the chance a student passes given the hours studied). If that probability is above a cut-off (usually 0.5), we say "pass"; otherwise "fail".

2.1 Why Not Just Use Linear Regression?

Linear regression outputs any number, from $-\infty$ to $+\infty$, but a probability must lie between 0 and 1. A straight line also reacts badly to extreme points and cannot model a "switch" from class 0 to class 1. We need a function that **squashes any number into** $(0, 1)$: the **sigmoid** (logistic) function.

2.2 The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = w_0 + w_1 x_1 + \cdots + w_n x_n = \mathbf{w}^\top \mathbf{x} + b.$$

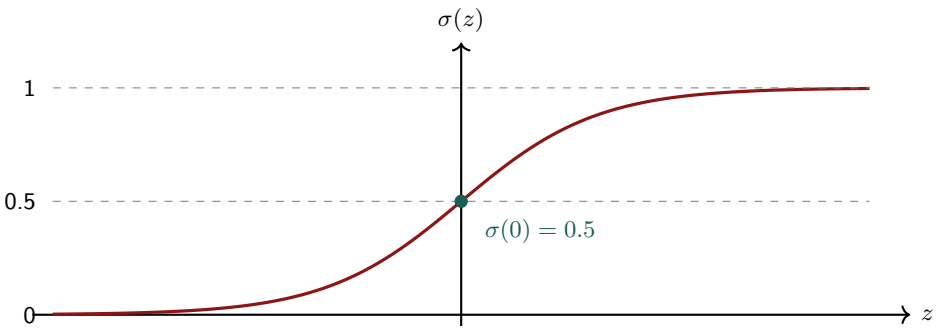


Figure 2. The sigmoid squashes any real number into the interval $(0, 1)$.

z	-3	-2	-1	**0**	**1**	**2**	**3**
$\sigma(z)$	0.047	0.119	0.269	0.500	0.731	0.881	0.953

Key properties: output always in $(0, 1)$; $\sigma(0) = 0.5$; symmetric: $\sigma(-z) = 1 - \sigma(z)$; smooth and differentiable with $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$.

2.3 The Model

$$P(y = 1 \mid x) = \hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq t \\ 0 & \text{otherwise} \end{cases} \quad \text{(default } t = 0.5\text{)}.$$

Because $\sigma(z) \geq 0.5$ exactly when $z \geq 0$, the **decision boundary** is the line (or plane) $\mathbf{w}^\top \mathbf{x} + b = 0$. Logistic regression is therefore a *linear classifier*.

Worked Example: Predicting pass/fail

Trained model: $z = -4 + 1 \cdot (\text{hours studied})$. A student studies $x = 5$ hours.
$z = -4 + 5 = 1 \;\Rightarrow\; \hat{p} = \sigma(1) = \frac{1}{1+e^{-1}} = 0.731$. Since $0.731 \geq 0.5$, predict **Pass**.
Boundary: $z = 0 \Rightarrow x = 4$ hours. Fewer than 4 hours gives $\hat{p} < 0.5$ (Fail); more gives Pass.

2.4 Odds and Log-Odds: Interpreting the Coefficients

$$\text{odds} = \frac{p}{1-p}, \qquad \text{log-odds (logit)} = \ln \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b.$$

So the model is *linear in the log-odds*. If $p = 0.8$ then odds $= 4$ ("4 to 1 in favour") and logit $= \ln 4 = 1.386$.

Increasing a feature x_j by one unit adds w_j to the log-odds, i.e. multiplies the odds by e^{w_j} (the **odds ratio**). For example, $w_j = 0.7 \Rightarrow e^{0.7} \approx 2.01$: each extra unit roughly doubles the odds of the positive class. A negative coefficient lowers the odds.

2.5 Learning the Weights: Log-Loss (Cross-Entropy)

Mean squared error is a poor choice with a sigmoid (non-convex). Instead we maximise the likelihood of the data, which is the same as minimising the **binary cross-entropy (log-loss)**:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i) \Big].$$

Intuition: if the true class is 1, the loss is $-\ln \hat{p}$; being confidently wrong is punished heavily.

True y	Predicted $\hat{p}$	Loss
1	0.90	$-\ln 0.9 = 0.105$ (good)
1	0.50	$-\ln 0.5 = 0.693$ (unsure)
1	0.10	$-\ln 0.1 = 2.303$ (confidently wrong)

Gradient descent

The gradients are remarkably simple:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i)\, x_{ij}, \qquad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i),$$

and each step updates $w_j \leftarrow w_j - \alpha\, \partial J / \partial w_j$.

Worked Example: One gradient step

Data: $(x = 1, y = 0)$ and $(x = 3, y = 1)$. Start $w = 0,\ b = 0,\ \alpha = 0.1$.
Both predictions: $\hat{p} = \sigma(0) = 0.5$. Errors $(\hat{p} - y) = 0.5$ and -0.5.
$\frac{\partial J}{\partial w} = \frac{1}{2}\big[(0.5)(1) + (-0.5)(3)\big] = \frac{1}{2}(0.5 - 1.5) = -0.5, \quad \frac{\partial J}{\partial b} = \frac{1}{2}(0.5 - 0.5) = 0.$
Update: $w = 0 - 0.1(-0.5) = 0.05, \quad b = 0$. The weight moves positive: more x now means higher probability of class 1, as the data suggest.

2.6 Choosing the Threshold

The default cut-off 0.5 is not sacred. In cancer screening, missing a patient is costly, so lower the threshold (say 0.3) to catch more positives (higher recall, lower precision). For a spam filter where wrongly blocking genuine mail is costly, raise it (higher precision). Section 5.6 shows this trade-off numerically.

2.7 Multiclass Logistic Regression

- **One-vs-Rest (OvR):** train one binary classifier per class ("is it class k or not?"); predict the class with the highest probability.

- **Softmax (multinomial) regression:** one model outputs probabilities for all K classes: $P(y = k \mid x) = \dfrac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$, and the probabilities sum to 1.

2.8 Regularisation

To avoid overfitting, especially with many features, add a penalty to the loss: **L2 (ridge)** $\lambda \sum w_j^2$ shrinks weights; **L1 (lasso)** $\lambda \sum |w_j|$ can set weights exactly to zero (feature selection). In scikit-learn the parameter `C` is the *inverse* of regularisation strength: smaller `C` means stronger regularisation.

2.9 Assumptions, Strengths and Weaknesses

Strengths	Weaknesses
Simple, fast, easy to implement and interpret	Only a linear decision boundary unless features are engineered
Gives calibrated-style probabilities	Struggles with highly non-linear patterns
Works well as a strong baseline	Sensitive to multicollinearity and outliers
Coefficients explain feature effects (odds ratios)	Needs feature scaling for good optimisation and regularisation
Regularisation is built in	Fails to converge cleanly on perfectly separable data without regularisation

Assumptions: the log-odds are a linear function of the features; observations are independent; little multicollinearity among features; a reasonably large sample size.

2.10 Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=42)
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=1000))
clf.fit(X_tr, y_tr)

proba = clf.predict_proba(X_te)[:, 1]        # P(y = 1)
pred  = (proba >= 0.5).astype(int)           # change 0.5 to move the threshold
print(clf[-1].coef_, clf[-1].intercept_)     # weights and bias
```

Watch Out

Logistic regression is named "regression" for historical reasons but **predicts classes**. Also, do not read $\hat{p}$ as a perfectly true probability unless the model has been checked for calibration.

Exam Tip

Typical questions: "Why sigmoid?", "Write the logistic regression hypothesis and cost function", "Differentiate linear and logistic regression". For the last one use a table: output (continuous vs probability/class), function (identity vs sigmoid), loss (MSE vs log-loss), use

(regression vs classification).

Pro Corner (for professionals)

Maximum-likelihood view. With $y_i \in \{0, 1\}$ the likelihood is $L = \prod_i \hat{p}_i^{y_i}(1 - \hat{p}_i)^{1-y_i}$; minimising $-\frac{1}{n} \ln L$ gives the log-loss above. The loss is **convex** in $(\mathbf{w}, b)$, so gradient descent reaches the global minimum. Second-order methods (Newton–Raphson, equivalently *iteratively reweighted least squares*) converge in a few iterations: $\mathbf{w} \leftarrow \mathbf{w} - H^{-1}\nabla J$ with Hessian $H = \frac{1}{n} X^\top S X$, $S = \mathrm{diag}(\hat{p}_i(1-\hat{p}_i))$. If classes are perfectly separable, the unregularised weights diverge to infinity, so always regularise. Logistic regression is the single-neuron special case of a neural network, and Gaussian Naïve Bayes with shared variances yields the same linear form, the two differing in how the weights are learned.

3 k-Nearest Neighbours (KNN)

In Plain Words

You move to a new neighbourhood and wonder whether people there prefer tea or coffee. You simply ask your **closest few neighbours** and go with the majority answer. KNN classifies a new sample exactly this way: *find the k most similar known samples and let them vote.*

Definition: KNN

A **lazy, instance-based, non-parametric** algorithm that classifies a new point by finding its k nearest training points (by a distance measure) and assigning the **majority class** among them. For regression it averages their target values.

3.1 The Algorithm

1. Choose k and a distance metric.
2. For the new point x_q, compute its distance to *every* training point.
3. Sort the distances and pick the k smallest (the k nearest neighbours).
4. **Classification:** take a majority vote of their classes. **Regression:** take the mean of their values.

"Lazy" means there is *no training phase*: the model is just the stored dataset, and all computation happens at prediction time.

3.2 Distance Measures

For two points $a = (a_1, \ldots, a_d)$ and $b = (b_1, \ldots, b_d)$:

Distance	Formula	Use
Euclidean (L_2)	$\sqrt{\sum_j (a_j - b_j)^2}$	Default for continuous features
Manhattan (L_1)	$\sum_j \lvert a_j - b_j \rvert$	Grid-like paths; robust to outliers
Minkowski	$\left(\sum_j \lvert a_j - b_j \rvert^p\right)^{1/p}$	General form: $p=1$ Manhattan, $p=2$ Euclidean
Hamming	Number of positions where values differ	Categorical / binary features
Cosine distance	$1 - \dfrac{a \cdot b}{\lVert a \rVert \, \lVert b \rVert}$	Text and high-dimensional sparse data

Worked Example: Distances between $a = (1, 2)$ and $b = (4, 6)$

Euclidean: $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$. Manhattan: $3 + 4 = 7$. Hamming for the categories (red, small, round) and (red, large, round): they differ in 1 position, so distance $= 1$.

3.3 Worked Example

Worked Example: Classify the query point $(3, 3)$

Training data:

Point	x_1	x_2	Class	Euclidean distance to $(3,3)$
A	1	1	Red	$\sqrt{4+4} = 2.828$
B	2	2	Red	$\sqrt{1+1} = 1.414$
C	4	3	Blue	$\sqrt{1+0} = 1.000$
D	5	4	Blue	$\sqrt{4+1} = 2.236$
E	1	3	Red	$\sqrt{4+0} = 2.000$

Sorted by distance: C (1.000, Blue), B (1.414, Red), E (2.000, Red), D (2.236, Blue), A (2.828, Red).

- $k = 1$: nearest is C $\Rightarrow$ **Blue**.
- $k = 3$: C, B, E = Blue, Red, Red $\Rightarrow$ **Red** (2 votes to 1).
- $k = 5$: three Red and two Blue $\Rightarrow$ **Red**.

Distance-weighted vote ($k = 3$, weight $= 1/d$): Blue: $1/1.000 = 1.000$. Red: $1/1.414 + 1/2.000 = 0.707 + 0.500 = 1.207$. Red wins.
Notice how the answer **changed with** k: choosing k matters.

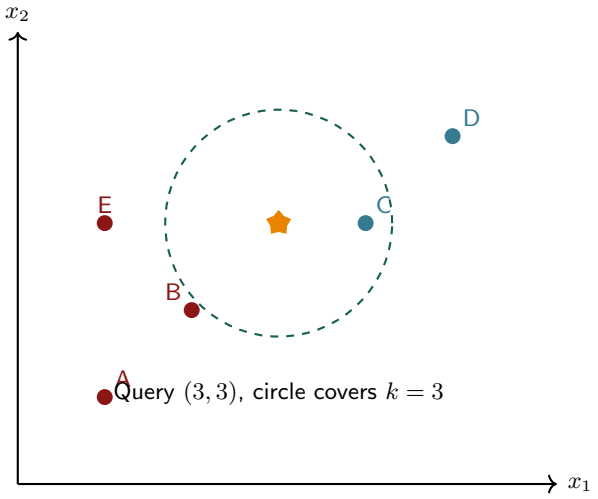


Figure 3. The three nearest neighbours of the query (star) are B, C and E: two red, one blue.

3.4 Choosing k

- **Small** k (e.g. 1): very flexible, follows noise: low bias, **high variance** (overfitting).
- **Large** k: smooth boundary, ignores local detail: high bias, low variance (underfitting). With $k = n$ every query gets the overall majority class.
- **Use an odd** k for two-class problems to avoid ties.
- **Rule of thumb:** $k \approx \sqrt{n}$ as a starting point; then choose the best k by **cross-validation**.

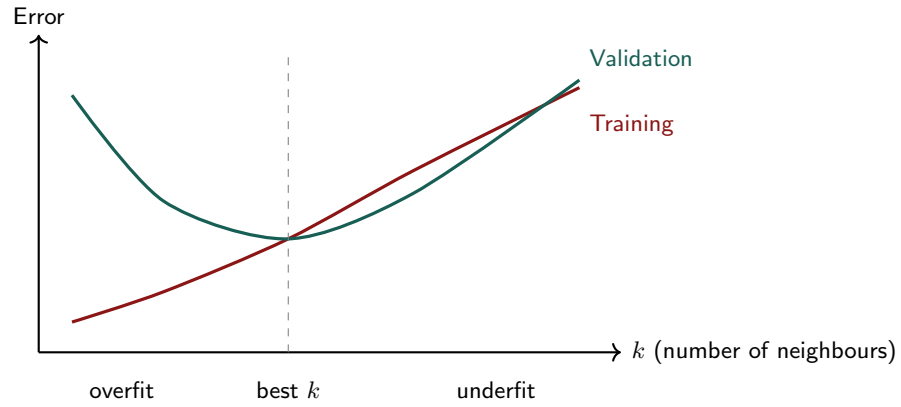


Figure 4. Typical error curves as k grows. Pick the k with the lowest validation error.

3.5 Weighted KNN

Closer neighbours should matter more. Give each neighbour a weight such as $w_i = 1/d_i$ (or $1/d_i^2$) and compare the total weight per class, as in the example above.

3.6 Feature Scaling is Essential

KNN is built on distances, so a feature with a large numeric range swamps the others. Consider $a = (\text{age } 25,\ \text{income } 50{,}000)$ and $b = (\text{age } 60,\ \text{income } 50{,}500)$. The age gap is 35 years but the income gap is 500, so $d = \sqrt{35^2 + 500^2} \approx 501$: the distance is almost entirely income. **Always scale** (min–max or standardise, using training statistics) and encode categories before using KNN.

3.7 Complexity

- Training: $O(1)$ (just store the data), but memory $O(nd)$.
- Prediction: $O(nd)$ per query by brute force, which is slow for large n. Speed-ups: KD-trees, ball trees, approximate nearest neighbour search (FAISS, Annoy).

Curse of dimensionality: as the number of features grows, all points become almost equally far apart, so "nearest" loses meaning. Use feature selection or PCA to keep d small.

3.8 Strengths and Weaknesses

Strengths	Weaknesses
Very simple, easy to understand	Slow predictions on big datasets
No training time; adapts instantly to new data	Large memory (stores everything)
Naturally handles multiclass	Sensitive to feature scale and irrelevant features
Non-linear, flexible decision boundary	Hurt by high dimensions and noisy data
Few assumptions about the data	Struggles with imbalanced classes (majority dominates the vote)

3.9 Code

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([("sc", StandardScaler()),
                 ("knn", KNeighborsClassifier(weights="distance"))])
grid = GridSearchCV(pipe,
        {"knn__n_neighbors": [1, 3, 5, 7, 9, 11],
         "knn__metric": ["euclidean", "manhattan"]},
        cv=5, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_, grid.score(X_test, y_test))
```

Watch Out

Never compute the scaler or choose k using the test set. Fit the scaler on training data inside a pipeline, and tune k with cross-validation on the training data only.

Exam Tip

KNN numericals are favourites: you will be given 4–6 points, a query, and asked to classify with $k = 3$. Show a distance table, sort it, mark the k nearest, and state the majority vote. Always write the Euclidean formula first and mention how a different k could change the result.

Pro Corner (for professionals)

Theory. As $n \to \infty$ the 1-NN classifier's error rate is at most about **twice the Bayes error rate** (Cover–Hart, 1967), and k-NN is consistent (approaches the Bayes optimum) when $k \to \infty$ and $k/n \to 0$. Practical engineering: use KD/ball trees for $d \lesssim 20$ and approximate indexes (HNSW, IVF) beyond that; learn a task-specific metric (Mahalanobis, metric learning) or embeddings so that Euclidean distance reflects semantic similarity; handle class imbalance by weighting votes by inverse class frequency; and note that KNN gives a crude probability estimate $\hat{P}(c \mid x) = \frac{\#\text{neighbours of class } c}{k}$.

4 Naïve Bayes

In Plain Words

A detective updates a suspicion as clues arrive: "The suspect owns a red car, was near the scene and has no alibi." Each clue raises or lowers the odds. Naïve Bayes does the same: it starts with how common each class is, then multiplies in how likely each observed feature would be *if* the sample belonged to that class, and finally picks the class with the highest overall score. It is "naïve" because it treats every clue as independent of the others.

4.1 Bayes' Theorem

Definition: Bayes' Theorem

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

For classification with class C and features x:

$$\underbrace{P(C \mid x)}_{\text{posterior}} = \frac{\overbrace{P(x \mid C)}^{\text{likelihood}}\ \overbrace{P(C)}^{\text{prior}}}{\underbrace{P(x)}_{\text{evidence}}}.$$

- **Prior** $P(C)$: how common the class is before seeing any features.
- **Likelihood** $P(x \mid C)$: how probable these features are within the class.
- **Evidence** $P(x)$: the same for every class, so it is dropped when we only compare classes.
- **Posterior** $P(C \mid x)$: our updated belief after seeing x.

Worked Example: Bayes' theorem with a medical test

A disease affects 1% of people. A test detects it 95% of the time if present (sensitivity) and gives a false positive 5% of the time if absent.
$P(\text{positive}) = 0.95(0.01) + 0.05(0.99) = 0.0095 + 0.0495 = 0.059.$
$P(\text{disease} \mid \text{positive}) = \dfrac{0.95 \times 0.01}{0.059} = \dfrac{0.0095}{0.059} \approx 0.161.$
Even after a positive test the chance of disease is only about **16%**, because the disease is rare (low prior). Priors matter!

4.2 The Naïve Assumption

With many features, $P(x_1, \ldots, x_n \mid C)$ is impossible to estimate directly (too many combinations). Naïve Bayes assumes the features are **conditionally independent given the class**:

$$P(x_1, \ldots, x_n \mid C) = \prod_{j=1}^{n} P(x_j \mid C).$$

Hence the classifier is

$$\hat{y} = \arg\max_{C}\ P(C) \prod_{j=1}^{n} P(x_j \mid C).$$

The assumption is rarely true (the words "free" and "offer" often occur together), yet the classifier often works well because it only needs the *ranking* of classes to be right, not exact probabilities.

4.3 Worked Example (Spam Filter)

Worked Example: Classify an email with "offer = Yes" and "link = Yes"

Training data: 10 emails.

Class	offer = Y	offer = N	link = Y	link = N	Total
Spam	4	2	4	2	6
Ham (not spam)	1	3	1	3	4

Priors: $P(\text{Spam}) = 0.6, \quad P(\text{Ham}) = 0.4.$
Likelihoods: $P(\text{offer=Y} \mid \text{Spam}) = \frac{4}{6}, \; P(\text{link=Y} \mid \text{Spam}) = \frac{4}{6}; \quad P(\text{offer=Y} \mid \text{Ham}) = \frac{1}{4}, \; P(\text{link=Y} \mid \text{Ham}) = \frac{1}{4}.$
Scores (unnormalised posteriors):

$$\text{Spam}: \; 0.6 \times \tfrac{4}{6} \times \tfrac{4}{6} = 0.6 \times 0.4444 = 0.2667, \qquad \text{Ham}: \; 0.4 \times \tfrac{1}{4} \times \tfrac{1}{4} = 0.4 \times 0.0625 = 0.0250.$$

Normalise: $P(\text{Spam} \mid x) = \dfrac{0.2667}{0.2667 + 0.0250} = \dfrac{0.2667}{0.2917} \approx 0.914, \;\; P(\text{Ham} \mid x) \approx 0.086.$
Decision: Spam, with about 91% probability.

4.4 The Zero-Frequency Problem and Laplace Smoothing

If a feature value never appears with a class in the training data, its likelihood is 0, and multiplying by 0 wipes out the whole product. The fix is **Laplace (add-one) smoothing**:

$$P(x_j = v \mid C) = \frac{\text{count}(v, C) + \alpha}{\text{count}(C) + \alpha\, k},$$

where k is the number of possible values of the feature and usually $\alpha = 1$.

Worked Example: Smoothing

Suppose "offer = Y" never occurred in the 4 ham emails: raw estimate $\frac{0}{4} = 0$. With smoothing ($k = 2$ values: Y, N): $\frac{0+1}{4+2} = \frac{1}{6} \approx 0.167$. The probability is small but no longer fatal.

4.5 Variants of Naïve Bayes

Variant	Feature type	Likelihood used
Gaussian NB	Continuous numbers (height, temperature)	Normal density with class mean μ_c and variance σ_c^2
Multinomial NB	Counts (word frequencies in text)	$P(w \mid c) = \dfrac{\text{count}(w, c) + \alpha}{\text{total words in } c + \alpha\lvert V\rvert}$
Bernoulli NB	Binary features (word present / absent)	Probability of presence per class
Categorical NB	Discrete categories	Category frequencies per class, as in the spam example

Gaussian Naïve Bayes

For a continuous feature, the likelihood in class c is

$$P(x \mid c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp\left(-\frac{(x - \mu_c)^2}{2\sigma_c^2}\right).$$

Worked Example: Density of height $x = 170$ for a class with $\mu = 165$, $\sigma^2 = 25$

$\sqrt{2\pi \cdot 25} = \sqrt{157.08} = 12.53$. Exponent: $-\frac{(170-165)^2}{2 \cdot 25} = -\frac{25}{50} = -0.5$, and $e^{-0.5} = 0.6065$. $P(170 \mid c) = \frac{0.6065}{12.53} \approx 0.0484$. (This is a density, not a probability; it is compared across classes.)

4.6 Practical Tips

- **Work with logs.** Multiplying many small probabilities underflows to 0. Compare $\ln P(C) + \sum_j \ln P(x_j \mid C)$ instead.
- **Remove or combine highly redundant features;** strong dependence breaks the independence assumption and makes probabilities over-confident.
- **Needs little data** and trains in a single pass over the data, so it is fast and scales to very large text collections.
- The probability outputs are often poorly calibrated; use them for *ranking*, or calibrate them (Platt scaling, isotonic regression).

4.7 Strengths and Weaknesses

Strengths	Weaknesses
Extremely fast to train and predict	Independence assumption rarely holds
Works with small datasets	Poor probability estimates
Handles many features, especially text, well	Cannot learn feature interactions
Naturally multiclass	Zero-frequency problem needs smoothing
Robust to irrelevant features	Gaussian variant assumes normal data

Applications: spam filtering, sentiment analysis, document and news categorisation, medical diagnosis, real-time prediction, recommender systems.

4.8 Code

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline

# numeric features
gnb = GaussianNB().fit(X_train, y_train)
print(gnb.predict_proba(X_test[:3]))

# text classification (spam)
spam_clf = make_pipeline(CountVectorizer(), MultinomialNB(alpha=1.0))
spam_clf.fit(train_texts, train_labels)
print(spam_clf.predict(["Win a free offer now, click the link"]))
```

Exam Tip

Expect: "Explain Bayes' theorem and the Naïve Bayes classifier with an example." Write the theorem, name prior/likelihood/posterior, state the independence assumption, then work a small numerical (like the spam table) ending with a clear **decision**. Mention Laplace smoothing for extra marks.

Pro Corner (for professionals)

MAP decision rule. Naïve Bayes is a *maximum a posteriori* classifier: $\hat{y} = \arg\max_C \left[\ln P(C) + \sum_j \ln P(x_j \mid C)\right]$. For Gaussian features with equal class variances the log-posterior ratio is *linear* in x, so the decision boundary is a hyperplane (the same functional form as logistic regression); with unequal variances it is quadratic. As a **generative** model it reaches its (higher) asymptotic error faster than the discriminative logistic regression, which tends to win with more data (Ng and Jordan, 2001). Add-α smoothing is equivalent to a Dirichlet prior on the multinomial parameters. Complement NB improves performance on imbalanced text data.

5 Confusion Matrix, Accuracy, Precision and Recall

In Plain Words

After training a classifier you must ask: *how good is it, and in what way does it go wrong?* A single score hides the story. The confusion matrix is a simple scoreboard that shows **every kind of right and wrong answer**, and from it we compute accuracy, precision and recall, each answering a different question.

5.1 The Confusion Matrix

Definition: Confusion Matrix

A table that compares the model's predicted classes with the true classes. For binary classification it is a 2×2 table with four counts.

		Predicted: Positive	Predicted: Negative
Actual	Positive	**TP** (true positive)	**FN** (false negative)
Actual	Negative	**FP** (false positive)	**TN** (true negative)

Cell	Meaning	Spam-filter example
TP	Predicted positive, actually positive	Spam correctly sent to the spam folder
TN	Predicted negative, actually negative	Genuine mail correctly kept in the inbox
FP (Type I error)	Predicted positive, actually negative: a *false alarm*	Genuine mail wrongly sent to spam
FN (Type II error)	Predicted negative, actually positive: a *miss*	Spam that reaches the inbox

Tip

Memory trick. The second word (positive/negative) is what the *model predicted*; the first word (true/false) says whether that prediction was *right*. "False Positive" = predicted positive, but wrong.

5.2 Accuracy, Precision and Recall

Accuracy

Definition: Accuracy

The fraction of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Question answered: "Overall, how often is the model right?"

Precision

Definition: Precision (Positive Predictive Value)

Of everything the model labelled positive, how much really is positive:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Question answered: "When the model says positive, can I trust it?" High precision means few false alarms.

Recall

Definition: Recall (Sensitivity, True Positive Rate)

Of all the real positives, how many the model found:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Question answered: "Did the model catch all the positives?" High recall means few misses.

	Precision	Recall
Denominator	All *predicted* positives ($TP + FP$)	All *actual* positives ($TP + FN$)
Punishes	False positives (false alarms)	False negatives (misses)
Analogy (fishing net)	Of the catch, how much is fish and not rubbish?	Of all the fish in the lake, how many did the net catch?
Important when	False alarms are costly: spam filter, recommending a job	Misses are costly: cancer or fraud screening

5.3 Worked Example

Worked Example: Disease screening on 200 patients

Results: $TP = 50,\ FN = 10,\ FP = 5,\ TN = 135.$

	Pred. Positive	Pred. Negative	Total
Actual Positive	50	10	60
Actual Negative	5	135	140
Total	55	145	200

$$\text{Accuracy} = \frac{50 + 135}{200} = \frac{185}{200} = 0.925$$

$$\text{Precision} = \frac{50}{50 + 5} = \frac{50}{55} = 0.909$$

$$\text{Recall} = \frac{50}{50 + 10} = \frac{50}{60} = 0.833$$

$$\text{Specificity} = \frac{135}{135 + 5} = \frac{135}{140} = 0.964$$

$$F_1 = \frac{2PR}{P + R} = \frac{2(0.909)(0.833)}{0.909 + 0.833} = 0.870 \quad \left(\text{also } \tfrac{2TP}{2TP+FP+FN} = \tfrac{100}{115}\right)$$

Interpretation: 92.5% of all patients are classified correctly; 91% of those flagged are truly

sick; but 1 in 6 sick patients (16.7%) is missed. For a serious disease, improving recall may be the priority.

5.4 The Accuracy Paradox

Worked Example: Why accuracy can mislead

1000 patients, only 10 have a rare disease. A lazy model says *"healthy"* for everyone. $TP = 0,\ FN = 10,\ FP = 0,\ TN = 990 \Rightarrow$ Accuracy $= \frac{990}{1000} = \mathbf{99}\%$, but Recall $= \frac{0}{10} = \mathbf{0}\%$. The model looks excellent yet finds *no* sick patient. On imbalanced data, rely on precision, recall and F_1.

5.5 More Metrics from the Same Matrix

Metric	Formula	Meaning
Specificity (TNR)	$\frac{TN}{TN+FP}$	Of real negatives, how many were correctly cleared
False positive rate	$\frac{FP}{FP+TN} = 1 - \text{Specificity}$	Fraction of negatives wrongly flagged
False negative rate	$\frac{FN}{FN+TP} = 1 - \text{Recall}$	Fraction of positives missed
Error rate	$\frac{FP+FN}{N} = 1 - \text{Accuracy}$	Overall mistake rate
F_1 **score**	$\frac{2PR}{P+R}$	Harmonic mean of precision and recall
F_β **score**	$\frac{(1+\beta^2)PR}{\beta^2 P+R}$	$\beta > 1$ favours recall; $\beta < 1$ favours precision
Balanced accuracy	$\frac{\text{Recall} + \text{Specificity}}{2}$	Accuracy that is fair to imbalanced classes
Matthews correlation (MCC)	Formula below	Single balanced score from -1 to $+1$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Why harmonic mean for F_1? The arithmetic mean can look fine when one value is terrible (precision 1.0, recall 0.0 gives mean 0.5). The harmonic mean punishes imbalance: that case gives $F_1 = 0$.

For the disease example, balanced accuracy $= \frac{0.833+0.964}{2} = 0.899$ and MCC $= \frac{50 \cdot 135 - 5 \cdot 10}{\sqrt{55 \cdot 60 \cdot 140 \cdot 145}} = \frac{6700}{8185} \approx 0.819$.

5.6 The Precision–Recall Trade-off and the Threshold

Most classifiers output a score (probability). The **threshold** converts it into a class. Moving the threshold trades precision against recall: lower it and you catch more positives (recall ↑) at the price of more false alarms (precision ↓), and vice versa.

Worked Example: Same model, two thresholds

Six samples with predicted probability and true class: $0.9\,(1),\ 0.8\,(1),\ 0.7\,(0),\ 0.6\,(1),\ 0.4\,(0),\ 0.2\,(0)$.
Threshold 0.5**:** predicted positive are $0.9, 0.8, 0.7, 0.6$. $TP = 3$ (0.9, 0.8, 0.6), $FP = 1$ (0.7), $FN = 0$, $TN = 2$. Precision $= \frac{3}{4} = 0.75$, Recall $= \frac{3}{3} = 1.00$.
Threshold 0.75**:** predicted positive are $0.9, 0.8$. $TP = 2$, $FP = 0$, $FN = 1$ (the 0.6 sample), $TN = 3$. Precision $= \frac{2}{2} = 1.00$, Recall $= \frac{2}{3} = 0.67$.
Raising the threshold made the model **more precise but less sensitive**.

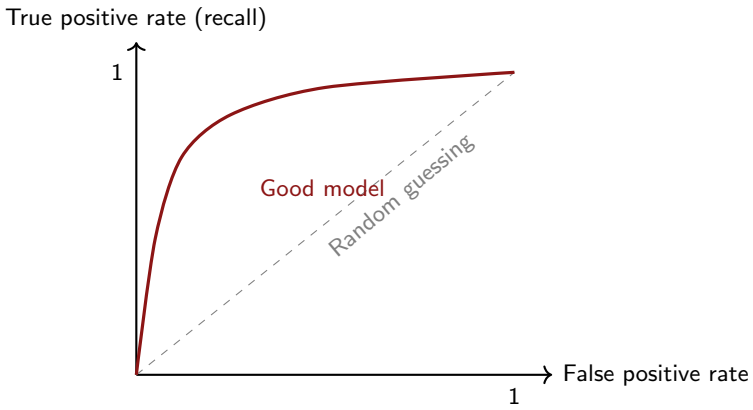


Figure 5. An ROC curve plots recall against false positive rate over all thresholds. The area under it (AUC) is 1 for a perfect model and 0.5 for random guessing.

5.7 Multiclass Confusion Matrix

For K classes the matrix is $K \times K$: rows are actual classes, columns are predicted classes, and the diagonal holds correct predictions. Precision and recall are computed **per class** (one-vs-rest): recall of class c = diagonal entry ÷ its row sum; precision of class c = diagonal entry ÷ its column sum.

Worked Example: Three-class example (100 samples)

	Pred A	Pred B	Pred C	Row total
Actual A	30	5	5	40
Actual B	4	25	1	30
Actual C	6	0	24	30
Column total	40	30	30	100

Accuracy $= \frac{30+25+24}{100} = 0.79$.
Class A: precision $= \frac{30}{40} = 0.75$, recall $= \frac{30}{40} = 0.75$. Class B: precision $= \frac{25}{30} = 0.833$, recall $= \frac{25}{30} = 0.833$. Class C: precision $= \frac{24}{30} = 0.80$, recall $= \frac{24}{30} = 0.80$.
Macro average (equal weight per class): precision $= \frac{0.75+0.833+0.80}{3} = 0.794$. **Micro average** (pool all counts) equals the accuracy, 0.79. **Weighted average** weights each class by its support (40, 30, 30).

5.8 Which Metric Should I Use?

Situation	Best metric	Reason
Balanced classes, equal error costs	Accuracy	Simple and adequate
Imbalanced classes	Precision, recall, F_1, PR-AUC	Accuracy is dominated by the majority class
Missing a positive is dangerous (cancer, fraud, security)	Recall (high sensitivity)	Catch every real case
False alarms are expensive (spam to inbox, legal action)	Precision	Trust positive predictions
Need one number balancing both	F_1 (or F_β)	Harmonic mean
Comparing models across thresholds	ROC-AUC or PR-AUC	Threshold-independent

5.9 Code

```
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, classification_report,
                             roc_auc_score, ConfusionMatrixDisplay)

y_pred = (proba >= 0.5).astype(int)

cm = confusion_matrix(y_test, y_pred)       # rows = actual, cols = predicted
tn, fp, fn, tp = cm.ravel()                 # binary case
print(cm)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1       :", f1_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, proba))
print(classification_report(y_test, y_pred))   # per-class precision/recall/F1
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
```

Watch Out

(1) Libraries differ in layout; scikit-learn puts *actual* on rows and *predicted* on columns, but some textbooks do the opposite, so check before computing. (2) Always evaluate on the **test set**, never on training data. (3) Define which class is "positive" explicitly.

Exam Tip

The most frequent exam question in this module: *"From the given confusion matrix compute accuracy, precision, recall (and F_1)"*. Steps: (i) label TP, FP, FN, TN; (ii) write each formula; (iii) substitute; (iv) give one line of interpretation. Do not forget to explain **what Type I and Type II errors** are.

Pro Corner (for professionals)

Probabilistic and cost-sensitive evaluation. Choose the threshold that minimises expected cost $= c_{FP} \cdot FP + c_{FN} \cdot FN$; for calibrated probabilities the optimal threshold is $t^* = \frac{c_{FP}}{c_{FP}+c_{FN}}$. Precision depends on the *prevalence* of positives (it is Bayes' theorem again: Precision $= \frac{\text{TPR}\cdot\pi}{\text{TPR}\cdot\pi+\text{FPR}\cdot(1-\pi)}$), which is why a test with excellent sensitivity and specificity still has low precision for rare conditions (see the 16% example in Chapter 4). ROC curves are insensitive to class imbalance, so under heavy imbalance prefer the **precision–recall curve** (average precision). Estimate confidence intervals for metrics via bootstrap or cross-validation; report calibration (reliability diagram, Brier score) when probabilities drive decisions.

6 Comparison and Revision Toolkit

6.1 Logistic Regression vs. KNN vs. Naïve Bayes

	Logistic Regression	KNN	Naïve Bayes
Idea	Sigmoid of a linear score gives $P(y=1 \mid x)$	Vote among the k closest training points	Bayes' rule with independent features
Type	Discriminative, parametric, eager	Instance-based, non-parametric, lazy	Generative, parametric, eager
Decision boundary	Linear	Flexible, non-linear	Linear or quadratic
Training cost	Moderate (iterative optimisation)	None (stores data)	Very low (counting)
Prediction cost	Very fast	Slow for large data	Very fast
Key hyperparameters	Regularisation strength `C`, penalty	k, distance metric, weights	Smoothing α, variant
Needs scaling?	Yes (recommended)	**Yes (essential)**	Not for categorical / count variants
Handles irrelevant features?	Moderately (use regularisation)	Poorly	Fairly well
Probabilities?	Good	Crude ($\frac{\text{votes}}{k}$)	Often over-confident
Interpretability	High (odds ratios)	High (neighbours)	High (class-conditional counts)
Best for	Baseline, tabular data, explainable models	Small, low-dimensional data with local structure	Text, small data, quick baselines

6.2 One-Page Summary

Topic	Remember this
Classification	Supervised learning with discrete labels: binary, multiclass, multilabel.
Logistic regression	$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$; linear boundary; log-loss; odds ratio e^w; threshold 0.5 by default; regularise.
KNN	Lazy; distance (Euclidean $\sqrt{\sum (a-b)^2}$); choose k by cross-validation; **scale features**; small k overfits, large k underfits.
Naïve Bayes	$P(C \mid x) \propto P(C) \prod P(x_j \mid C)$; conditional independence; Laplace smoothing; Gaussian / Multinomial / Bernoulli.
Confusion matrix	TP, TN, FP (Type I), FN (Type II); rows actual, columns predicted.
Accuracy	$\frac{TP+TN}{N}$: overall correctness; misleading on imbalanced data.
Precision	$\frac{TP}{TP+FP}$: trust in positive predictions.
Recall	$\frac{TP}{TP+FN}$: coverage of real positives.
F_1	Harmonic mean of precision and recall.

6.3 Formula Sheet

Name	Formula
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$
Log-odds	$\ln \dfrac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b$
Log-loss	$-\frac{1}{n}\sum \left[y \ln \hat{p} + (1-y)\ln(1-\hat{p})\right]$
Gradient	$\frac{\partial J}{\partial w_j} = \frac{1}{n}\sum(\hat{p}-y)x_j$
Euclidean / Manhattan	$\sqrt{\sum(a_j-b_j)^2}$ / $\sum \lvert a_j - b_j \rvert$
Bayes' theorem	$P(A \mid B) = \dfrac{P(B \mid A)P(A)}{P(B)}$
Naïve Bayes	$\hat{y} = \arg\max_C P(C)\prod_j P(x_j \mid C)$
Laplace smoothing	$\dfrac{\text{count}+1}{\text{total}+k}$
Accuracy	$\dfrac{TP+TN}{TP+TN+FP+FN}$
Precision / Recall	$\dfrac{TP}{TP+FP}$ / $\dfrac{TP}{TP+FN}$
Specificity	$\dfrac{TN}{TN+FP}$
F_1	$\dfrac{2PR}{P+R} = \dfrac{2TP}{2TP+FP+FN}$

6.4 Commonly Confused Pairs

Pair	First	Second
Linear vs logistic regression	Predicts a number; MSE	Predicts probability/class; log-loss
Precision vs recall	Correct among predicted positives	Found among actual positives
Type I vs Type II error	False positive (false alarm)	False negative (miss)
KNN vs k-Means	Supervised classifier	Unsupervised clustering
Lazy vs eager learner	Stores data, works at prediction	Builds a model during training
Generative vs discriminative	Models $P(x, y)$ (Naïve Bayes)	Models $P(y \mid x)$ (logistic regression)
Prior vs posterior	Belief before seeing features	Belief after seeing features
Accuracy vs balanced accuracy	Overall correctness	Average of recall and specificity

6.5 Short-Answer Practice

1. **Define classification.** Supervised task of assigning inputs to one of a finite set of classes learned from labelled examples.
2. **Why is logistic regression called a classifier?** It outputs a probability via the sigmoid, which is thresholded to give a class.
3. **State the sigmoid function and its range.** $\sigma(z) = 1/(1+e^{-z})$; range $(0, 1)$.

4. **What loss does logistic regression minimise?** Binary cross-entropy (log-loss).
5. **What does KNN stand for and why is it lazy?** k-Nearest Neighbours; it does no training and defers all computation to prediction.
6. **How is k chosen?** By cross-validation; odd values for binary problems; start near $\sqrt{n}$.
7. **Why scale features for KNN?** Distances are dominated by large-range features otherwise.
8. **State Bayes' theorem.** $P(A \mid B) = P(B \mid A)P(A)/P(B)$.
9. **Why is Naïve Bayes "naïve"?** It assumes all features are conditionally independent given the class.
10. **What is Laplace smoothing?** Adding a pseudo-count (usually 1) to every count so no probability is zero.
11. **What is a confusion matrix?** A table of predicted vs actual class counts: TP, TN, FP, FN.
12. **Differentiate precision and recall.** Precision $= TP/(TP+FP)$; recall $= TP/(TP+FN)$.
13. **When is accuracy a poor metric?** With imbalanced classes.
14. **What is F_1?** The harmonic mean of precision and recall.

6.6 Long-Answer Outlines

Question	Skeleton that earns full marks
Explain logistic regression	Why not linear regression; sigmoid and hypothesis; decision boundary; log-loss and gradient descent; numerical prediction; threshold; strengths and weaknesses.
Explain the KNN algorithm with an example	Steps; distance formulas; distance table for a query; effect of k; scaling; complexity; advantages and limitations.
Explain Naïve Bayes with an example	Bayes' theorem; independence assumption; prior/likelihood/posterior; worked spam table; Laplace smoothing; variants.
Explain confusion matrix and derived metrics	2×2 table; TP/FP/FN/TN; accuracy, precision, recall, F_1 formulas; numerical; accuracy paradox; when to use which.
Compare the three classifiers	Use the comparison table: idea, boundary, speed, scaling, probabilities, best use.

6.7 Multiple-Choice Practice

1. The output of the sigmoid function lies in: (a) $[-1, 1]$ (b) $(0, 1)$ (c) $(0, \infty)$ (d) $(-\infty, \infty)$
2. Logistic regression has a: (a) linear decision boundary (b) circular boundary (c) tree-shaped boundary (d) no boundary
3. Loss function of logistic regression: (a) MSE (b) hinge (c) log-loss (d) absolute error
4. $\sigma(0)$ equals: (a) 0 (b) 0.5 (c) 1 (d) e
5. KNN is called lazy because: (a) it is slow to code (b) it has no explicit training phase (c) it ignores data (d) it uses few features
6. In KNN, a very small k typically causes: (a) underfitting (b) overfitting (c) no change (d) faster training
7. The most important preprocessing step for KNN: (a) feature scaling (b) taking logs (c) removing rows (d) adding noise

8. Manhattan distance between $(1, 2)$ and $(4, 6)$ is: (a) 5 (b) 7 (c) 3 (d) 25
9. Naïve Bayes assumes features are: (a) identical (b) conditionally independent given the class (c) correlated (d) Gaussian only
10. Laplace smoothing solves: (a) overfitting (b) zero-frequency problem (c) scaling (d) multi-collinearity
11. Which is a generative classifier? (a) logistic regression (b) KNN (c) Naïve Bayes (d) linear regression
12. A false positive is also called: (a) Type I error (b) Type II error (c) true negative (d) recall
13. Precision = (a) $\frac{TP}{TP+FN}$ (b) $\frac{TP}{TP+FP}$ (c) $\frac{TN}{TN+FP}$ (d) $\frac{TP+TN}{N}$
14. Recall = (a) $\frac{TP}{TP+FN}$ (b) $\frac{TP}{TP+FP}$ (c) $\frac{FP}{FP+TN}$ (d) $\frac{TN}{TN+FN}$
15. If $TP = 40, FP = 10, FN = 20, TN = 130$, accuracy is: (a) 0.80 (b) 0.85 (c) 0.90 (d) 0.75
16. With the same numbers, precision is: (a) 0.67 (b) 0.80 (c) 0.85 (d) 0.20
17. Missing a cancer patient is a: (a) false positive (b) false negative (c) true positive (d) true negative
18. The harmonic mean of precision and recall is: (a) accuracy (b) specificity (c) F_1 score (d) AUC

Tip

Answers: 1-b, 2-a, 3-c, 4-b, 5-b, 6-b, 7-a, 8-b, 9-b, 10-b, 11-c, 12-a, 13-b, 14-a, 15-b, 16-b, 17-b, 18-c.

6.8 Glossary

Term	Meaning
Class / label	A category the model predicts
Decision boundary	Surface separating regions assigned to different classes
Odds	$p/(1-p)$
Logit	Log of the odds; the linear score in logistic regression
Sigmoid	S-shaped function mapping reals to $(0, 1)$
Log-loss	Cross-entropy cost for probabilistic classifiers
Threshold	Probability cut-off that turns a score into a class
Lazy learner	Algorithm that postpones computation until prediction
Neighbour	One of the k closest training points to a query
Prior	Probability of a class before seeing features
Likelihood	Probability of features given the class
Posterior	Probability of the class given the features
Smoothing	Adding pseudo-counts to avoid zero probabilities
Confusion matrix	Table of actual vs predicted class counts
Type I error	False positive
Type II error	False negative
Sensitivity	Another name for recall
Specificity	True negative rate
ROC / AUC	Curve of TPR vs FPR; area under it summarises ranking quality

End of Module 3 — Classification Techniques
Course instructor: Prof. Rajesh Nag